Cinematic Editor - Full Handoff

Cinematic Editor — Session Handoff

Prepared 2026-11-08 for a new Claude Code session. Read this first, then PLAN.md, then KICKOFF.md. All three are in this document.

1. What this project is

A prompt-driven editing platform for real cinematic footage. Owner: Surya (suryaobb on GitHub). Two product lines on one engine:

- **Postcard mode**: 15–90 s music-driven, color-graded reels for hotels and travel.
- **Vlog mode**: 4–20 min transcript-driven narrative edits of the owner building a cabin and making jewelry.

Desktop is the workstation; the phone (installable PWA) is for browsing, prompting, reviewing drafts, and approving renders. Real footage only; no AI-generated video in v1.

2. State of the work so far

- A full product and technical plan exists (PLAN.md, Part B below).
- A kickoff package exists with a CLAUDE.md template and per-phase prompts (KICKOFF.md, Part C below).
- Both were committed to branch `claude/cinematic-video-editing-plan-m6oir3` in `suryaobb/trading-suite` but **could not be pushed**: the Claude GitHub App has no access to that repository. Fix before any coding session, at https://claude.ai/connect-github or https://github.com/apps/claude/installations/select_target.
- Decision taken: the product gets **its own repository** (suggested name `cinematic-editor`), not trading-suite.
- Decision taken: build **Postcard mode first**.

3. What is known about the owner's footage

Found in the owner's Google Drive (connector was attached to the planning session):

Folder / files	Camera (inferred)	Notes
`DJI_20241018194xxx_06xx_D.MP4` (Oct 2024, ~40–70 MB each)	DJI drone	`_D` suffix usually means D-Log; confirm with ffprobe (color transfer / `color_trc`).
`Hotel + Spa.mov`, `SPA.mov`, `Hedonist's Guide.mov`, `New-Hedonist.mov` (Feb 2025, 10–30 MB)	Unknown, owner to confirm	Likely already-edited exports; check whether they are source or finished pieces.

Open questions the owner still needs to answer (from PLAN.md section 7): cameras and log profiles for each line, footage location and volume, desktop GPU, finishing path (ffmpeg vs DaVinci Resolve), deliverable aspect

ratios and captions, music library and licensing.

4. Intake paths for footage

Google Drive (works from a phone)

- Owner creates one folder, e.g. "Cinematic Editor Inbox", shares it as "anyone with the link, viewer", and uploads clips.
- The Drive connector lists files, but its download tool returns base64 inline, which is unusable for video. Download with curl instead: `https://drive.google.com/uc?export=download&id=<fileId>` (large files redirect to `drive.usercontent.google.com` and may need the confirm-token dance).
- A personal cloud environment named **cinematic-editor** was created and approved. It allows `drive.google.com` and `drive.usercontent.google.com`, keeps package registries and MCP servers reachable, and has a setup script `pip install -q scenedetect opencv-python-headless`. The script is unproven until a fresh session starts on that environment. Start new sessions on it.

Remote Control (best for real volume)

- In the Claude desktop app: Settings → Claude Code → Remote Control → add the footage folder. A cloud session can then attach to that folder and run ffmpeg, Whisper, and labeling on the owner's machine with the originals in place.
- A cloud session attaches to one folder at a time and stays tied to the first computer it attaches to. Multiple Remote Control sessions can run on one computer; use worktree mode for parallel code sessions.

Cloud container facts (planning session): ffmpeg 6.1, Python 3, 4 CPUs, 15 GB RAM, no GPU, ~30 GB free disk. Fine for tens of clips per day, not terabytes.

5. Day-one deliverable ("Phase 1 lite")

Before the full stack, produce something the owner can review:

1. Pull every clip; ffprobe metadata; detect camera and log profile; split long recordings into shots (PySceneDetect).
2. 720p proxy, thumbnails, and a contact sheet per shot.
3. Label each shot with a vision model using the controlled vocabulary in PLAN.md (subject, place, time of day, weather, camera move, shot size, mood, people, text on screen) plus a quality score; group near-duplicate takes.
4. Output `catalog.json` + `catalog.csv` and one self-contained `library.html` with thumbnails, filters by label and quality, and search.
5. Review labels with the owner and fix the vocabulary. That becomes ground truth for the real catalog in Phase 1.

6. How to run the build

Recommended: a Claude Code Project at claude.ai/code with the new repo attached, one thread per phase. If Projects is not available on the account, one cloud session per phase on the cinematic-editor environment, pasting the phase prompt from KICKOFF.md. Use a Remote Control thread for anything that needs real footage or the desktop GPU. A regular claude.ai Project (mobile app) is fine for planning and prompt writing, but cannot build.

7. Working rules carried over

- The EDL JSON is the contract between the edit agent and the renderer; the agent may only reference shot IDs returned by its tools, and a validator rejects anything else.
- Heavy work always runs in a worker job, never in the API process.
- Phone only ever sees proxies/HLS, never originals.
- Each phase ends with a runnable demo and a `docs/phases/<n>.md` note.
- Ask before adding any paid service beyond the Anthropic API.
- Never put model identifiers in commits, PR text, or code comments.

Part B — PLAN.md

Status: **draft for review** (v0.1, 2026-10-12)

A prompt-driven editing system that turns a labeled library of cinematic clips into finished, color-graded videos. Two product lines share one engine:

Line	Example	Length	Audio	Feel
Postcard	Hotel reel, destination teaser, travel montage	15 s – 90 s	Music-driven, no dialogue	Slow, graded, beat-cut, logo end card
Vlog	Cabin build week 3, making a silver ring	4 – 20 min	Talking + b-roll + music bed	Narrative, transcript-driven, chaptered

Desktop is the primary workstation. The phone is for browsing the library, sending prompts, reviewing drafts, and approving renders.

---

1. Goals and non-goals

Goals

1. Connect a database of clips (local drive, NAS, or cloud bucket) and have every clip automatically proxied, labeled, transcribed, and searchable.
2. Type a prompt ("30 s golden-hour pool reel for Hotel X, warm look, slow") and get a draft edit with music, grade, titles, and an end card.
3. Apply a consistent, repeatable color grade per client or per channel, including correct handling of log footage.
4. Edit long-form vlogs from the transcript: pull selects, cut silences, drop in matching b-roll, add chapters and captions.
5. Review and approve from a phone; render on the desktop or a worker.
6. Hand off to DaVinci Resolve when a project needs a human finishing pass.

Non-goals (for v1)

- A full manual NLE with frame-accurate keyframing. We generate edits; fine surgery happens in Resolve via export.
- Generating footage with AI. Real footage is the product. (Optional later: AI upscaling and sky/plate fixes.)
- Multi-tenant SaaS from day one. Build for one operator with multiple client workspaces; keep the data model ready for tenants.

---

2. User workflows

2.1 Ingest and organize ("sort through")

1. Point the app at a source: a folder, an external drive, or an S3/R2 bucket.
2. The ingest worker, per file:

    - reads metadata with ffprobe (codec, resolution, fps, duration, camera, color space/transfer, GPS/date if present);
    - detects scene/shot boundaries so one long recording becomes many "shots";
    - renders a 720p H.264 proxy (for preview and AI) and an HLS version for phone;
    - extracts thumbnails and a contact sheet per shot;
    - transcribes speech (Whisper) with word timestamps, flags silence;
    - labels each shot with a vision model: subject, location type, time of day, weather, camera move (static / pan / gimbal / drone / handheld), shot size (wide / medium / close / detail), mood, people present, text on screen;
    - scores technical quality: sharpness, exposure clipping, stability, noise;
    - computes a visual embedding per shot for semantic search ("misty lake at dawn");
    - detects near-duplicates (ten takes of the same pan) and groups them.
3. Library view: filter by project, client, label, date, quality score; search by text; "find similar" from any shot; batch-tag; mark favorites and rejects.

2.2 Define a Look (grade)

A **Look** is a named, versioned recipe:

- input transform per camera (e.g. S-Log3 → Rec.709, D-Log → Rec.709, Apple Log → Rec.709);
- creative 3D LUT (.cube) and strength;
- lift/gamma/gain, saturation, contrast, highlight roll-off, grain, vignette;
- optional reference frame for automatic shot matching (match exposure and white balance across clips before the creative LUT);
- typography, palette, logo, lower-third style, end-card template;
- pacing defaults (average shot length, transition type), preferred aspect ratios, music mood tags.

Looks belong to a **Workspace** (e.g. "Hotel Aurelia", "Cabin Vlog", "Jewelry").

2.3 Prompt → edit (Postcard mode)

1. Choose workspace, pick or auto-select music, write the prompt.
2. The edit agent (an LLM with tools against the catalog) produces a plan: target duration, aspect, section structure (hook → establishing → details → people/food → signature moment → end card), then searches the catalog section by section with constraints (quality ≥ threshold, no near-duplicates, variety of shot sizes, time of day continuity).
3. Beat detection on the music gives cut points; shots are trimmed to the best interval (highest motion/quality moment) and aligned to beats.
4. Output is an **EDL** (edit decision list, JSON) — not a video yet. The EDL is the contract between brain and renderer.
5. Fast proxy render (seconds) for review → tweak via chat ("swap the third shot for a drone shot", "slower", "more people") → final render at full resolution.

2.4 Prompt → edit (Vlog mode)

1. Prompt describes the story: "Week 3 of the cabin, focus on the roof framing, keep the part where I explain the rafter math, cut anything over 12 minutes."
2. The agent works from the transcript: scores each spoken segment for relevance and energy, removes silences/filler, builds a spoken spine, then inserts b-roll whose labels/embeddings match what's being said at that moment.

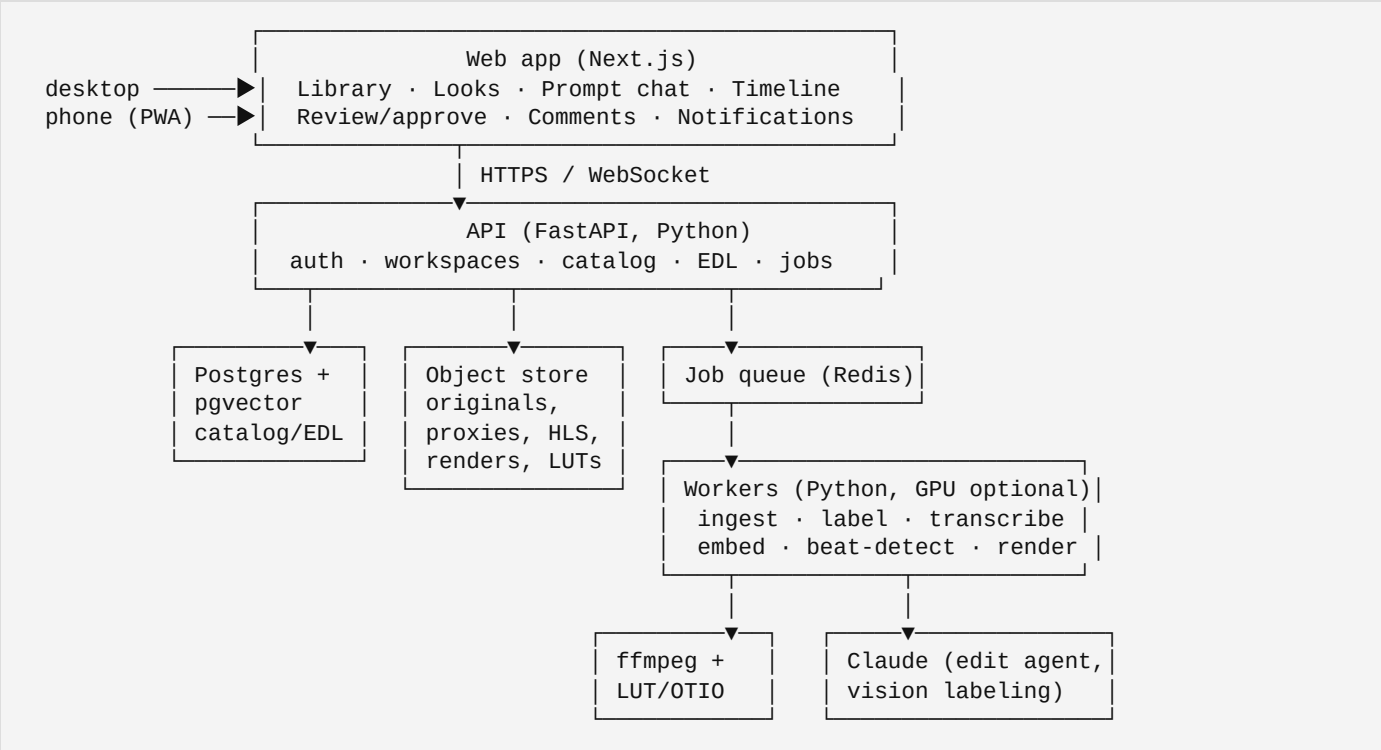
3. Adds chapters, burned or sidecar captions, music bed with ducking under speech, intro/outro from the Look.
4. Same review loop; export OTIO/FCPXML to Resolve for a manual pass when wanted.

2.5 Review and approve (phone)

- Watch the HLS proxy of a draft, scrub, leave timestamped comments ("too dark", "replace this").
- Comments are fed back to the agent as revision prompts.
- Approve → final render queued → notification with download/share link.

---

3. System architecture



Deployment shape for v1: everything runs on the desktop machine (Docker Compose), footage on a local/NAS drive, and the web app is reachable from the phone over Tailscale (or a small cloud VM fronts the API and serves proxies from R2 if you want access away from home). Cloud GPU is optional and only needed if the desktop lacks one.

3.1 Component choices

Concern	Choice	Why
Backend	Python 3.12, FastAPI	Best ecosystem for ffmpeg, Whisper, PySceneDetect, librosa, OpenTimelineIO; matches your existing Python tooling
Frontend	Next.js + React + Tailwind, installable PWA	One codebase for desktop and phone; HLS playback works everywhere
Catalog DB	Postgres + pgvector	Relational catalog plus vector search in one place
Object store	Local disk / NAS for v1; S3-compatible (Cloudflare R2) when remote access is needed	Cheap, no egress fees, works with every tool

Concern	Choice	Why
Queue	Redis + RQ (or Celery)	Simple, Python-native, visible job state
Shot detection	PySceneDetect	Robust content-aware cuts
Transcription	faster-whisper (local GPU/CPU)	Word timestamps, offline, free
Visual labels	Claude vision on sampled frames (Haiku-class for bulk, Sonnet/Opus-class for hard shots); optional Gemini video input for long clips	Structured JSON labels, cheap at scale
Embeddings	SigLIP/CLIP (open-source, local)	Fast "find similar" and text-to-shot search without API cost
Quality scoring	OpenCV (Laplacian sharpness, histogram clipping, optical-flow jitter)	Deterministic, free
Beat detection	librosa (onset/tempo)	Good enough for cut alignment
Edit agent	Claude (Fable/Opus-class) with tool use over the catalog	Reasoning + constrained tool calls; never invents clip IDs
Render	ffmpeg filtergraph: `lut3d`, `eq`, `curves`, `zoompan`, `xfade`, `drawtext`/ASS subtitles, `sidechaincompress` for ducking	Deterministic, scriptable, headless
Overlays/titles	ASS subtitles via ffmpeg for v1; Remotion (React) later for motion graphics	Start simple, upgrade when designs need motion
NLE handoff	OpenTimelineIO → FCPXML / Resolve import; optional Resolve scripting API for grade application	Keeps a professional finishing path

3.2 Data model (core tables)

- `workspace` — client or channel; owns looks, music, assets.
- `asset` — one source file: path/URL, checksum, ffprobe metadata, camera, color space, capture time, GPS, ingest status.
- `shot` — segment of an asset: in/out, proxy URL, thumbnails, quality scores, motion stats, duplicate-group id, embedding (vector), labels (JSONB), favorite/reject flags.
- `transcript_segment` — asset, start/end, text, words with timestamps, speaker, energy score.
- `label_vocab` — controlled vocabulary (subject, place, time-of-day, move, size, mood) so the agent searches with known terms.
- `look` — versioned grade + brand recipe (see 2.2), LUT file references.
- `music_track` — file, BPM, beats array, mood tags, license note.
- `project` — workspace, brief/prompt history, target specs.
- `edit` — project, version, EDL JSON, agent reasoning summary, status.
- `render` — edit version, preset (proxy/final), aspect, output URL, duration, logs.
- `comment` — edit version, timestamp, author, text, resolved flag.
- `job` — type, payload, status, progress, worker, error.

3.3 EDL format (the brain ↔ renderer contract)

```
{
  "version": 1,
  "project_id": "…",
  "look_id": "…",
  "output": { "aspect": "9:16", "width": 1080, "height": 1920, "fps": 24, "duration": 30.0
```

```
},
          "music": { "track_id": "…", "in": 12.4, "gain_db": -3, "duck_under_speech": true },
          "tracks": {
            "video": [
              { "shot_id": "…", "in": 3.2, "out": 5.9, "speed": 1.0, "reframe": "auto-subject",
                "transition_in": { "type": "cut" }, "grade_override": null }
            ],
            "speech": [ { "asset_id": "…", "in": 41.0, "out": 52.3 } ],
            "titles": [ { "t": 0.5, "dur": 2.5, "style": "hero", "text": "Aurelia, Santorini" } ],
            "end_card": { "template": "logo-fade", "dur": 2.0 }
          }
        }
```

Human-readable, diffable, and every ID must exist in the catalog (validated before render). Vlog edits add `chapters` and `captions`.

3.4 Edit agent design

- Tools: `search_shots(query, filters, k)`, `similar_shots(shot_id)`, `get_shot(id)`, `search_transcript(query)`, `music_beats(track_id)`, `propose_edl(edl)`, `validate_edl(edl)`, `render_preview(edl)`.
- The agent cannot reference a shot it did not retrieve via a tool call; the validator rejects unknown IDs and duplicate-group collisions.
- A **style card** per workspace (pacing, shot-size mix, do/don't list learned from your approvals and rejections) is injected into the prompt.
- Every revision is a new EDL version; comments on a draft are translated into constraints for the next version.
- Cost control: catalog search runs locally (pgvector); the LLM only sees compact shot summaries (ID, labels, scores, duration), not frames, during assembly.

3.5 Color pipeline

1. On ingest, detect the camera transfer function (metadata, filename pattern, or user mapping per camera profile).
2. Normalize: apply the camera's technical LUT to a working space (Rec.709 for v1; ACES later if needed).
3. Match: optional per-shot exposure/white-balance alignment to the Look's reference frame (histogram/mean-std matching on the proxy, then applied as `eq`/`colorbalance` params to the full-res).
4. Create: apply the Look's creative LUT at its strength, then grain/vignette.
5. Preview grades on the proxy with a side-by-side/wipe compare; batch-apply to a selection; "re-grade whole project" is a single job.
6. Export the resolved per-shot grade as metadata so Resolve import carries it.

3.6 Mobile strategy

- Installable PWA, responsive layouts; no native app in v1.
- Phone gets: library browsing (thumbnails + HLS proxies), search, favorites, prompt chat, draft review with timestamped comments, approve/reject, push notifications via web push when a render finishes.
- Phone never touches originals; all heavy work is queued to the desktop/worker.
- If a native app becomes worthwhile (camera-roll upload, offline), wrap with Capacitor later without changing the web code.

---

4. Phased roadmap

Each phase ends with something you use for real.

Phase 0 — Decisions and setup (1 week)

- Confirm cameras and formats (log profiles?), footage volume, where footage lives.
- Stand up repo, Docker Compose (Postgres, Redis, API, worker, web), storage layout.
- Collect 2–3 hours of representative footage from each line (hotel, cabin, jewelry).

Phase 1 — Ingest, catalog, search (2–3 weeks)

- ffprobe + shot detection + proxies + thumbnails + HLS.
- Vision labeling, transcription, embeddings, quality scores, duplicate grouping.
- Library UI with filters, text/semantic search, favorites/rejects, batch tagging.
- **Done when:** you can ask "misty lake drone shots, sharp, dawn" and get the right shots.

Phase 2 — Looks and grading (2 weeks)

- Look model, LUT upload, camera profiles, preview with compare wipe.
- ffmpeg grade pipeline on proxies and full-res; batch re-grade.
- Shot matching to a reference frame.
- **Done when:** a hotel look applied across mixed clips reads as one consistent piece.

Phase 3 — Postcard mode (3 weeks)

- Music library with beat detection.
- Edit agent with catalog tools, EDL validator, proxy renderer, final renderer (9:16, 16:9, 4:5, 1:1 with auto-reframe).
- Titles/end cards via Look templates.
- Chat-based revision loop; edit version history.
- **Done when:** one prompt produces a reel you would post for a hotel with ≤ 2 revision rounds.

Phase 4 — Vlog mode (3 weeks)

- Transcript-driven select scoring, silence/filler removal, b-roll matching.
- Chapters, captions (burned + .srt), music ducking, intro/outro.
- OTIO/FCPXML export to Resolve.
- **Done when:** a week of cabin footage becomes a reviewable 10-minute cut in under an hour of your time.

Phase 5 — Mobile review and remote access (1–2 weeks)

- PWA polish, HLS review player, timestamped comments → revision prompts, web push.
- Tailscale or cloud-fronted access; R2 for proxies if away from home.
- **Done when:** you can approve a render from your phone on the road.

Phase 6 — Learning and scale (ongoing)

- Style cards learned from approvals/rejections; A/B drafts.
- Multi-workspace client management, deliverable presets per platform.
- Optional: Remotion motion graphics, Resolve scripting for grade handoff, AI upscale for older footage, direct publishing to Instagram/TikTok/YouTube.

---

5. Cost and performance notes

Rough per-hour-of-footage processing on a desktop with a mid-range GPU:

Step	Time	External cost
Proxies + thumbnails + shot detection	~10–15 min	none
Transcription (faster-whisper, GPU)	~3–5 min	none
Embeddings + quality scores	~2–3 min	none
Vision labeling (1 frame / shot, ~300 shots)	~5 min	low single-digit dollars with a Haiku-class model
Edit planning per draft	~30–90 s	cents to tens of cents per draft
Final render (1080p, 30 s reel)	~1–2 min	none

Storage: originals dominate; proxies add roughly 5–8 % of original size. Plan for a dedicated drive or bucket per year of footage.

---

6. Risks and mitigations

Risk	Mitigation
Agent picks technically fine but boring shots	Quality + "interest" scores (motion, faces, contrast), variety constraints, style card from your approvals
Grades look different per camera	Camera profiles with technical LUTs; shot matching to a reference frame; always preview on proxies first
ffmpeg renders look less polished than an NLE	Keep titles simple in v1; OTIO export for finishing; Remotion later for motion graphics
Long log clips bloat labeling cost	Label per shot (not per second), sample 1–3 frames, use a cheap model first and escalate only on low confidence
Phone playback of large files	Only HLS proxies on mobile; originals never leave the desktop/bucket
Music licensing for hotel deliverables	Music library tracks carry a license field; agent only picks licensed tracks for client workspaces
Hallucinated clip references	Tool-constrained agent + EDL validator

---

7. Decisions needed from you

1. **Cameras and formats.** Which cameras (and log profiles) shoot the hotel work, the cabin, and the jewelry? This determines the color pipeline.
2. **Where the footage lives.** Local drive/NAS at the desktop, or already in a cloud bucket? Roughly how many TB now and per year?
3. **Desktop hardware.** GPU? This decides whether Whisper/embeddings run locally or on a cloud worker.
4. **Finishing path.** Is "good enough to post" from ffmpeg acceptable for hotel deliverables, or do you expect to finish every client piece in Resolve?
5. **Deliverable specs.** Which platforms/aspect ratios matter first (Reels 9:16, YouTube 16:9, both)? Captions required on vlogs?
6. **Music.** Do you have a licensed library (Artlist/Epidemic) or should the system manage one?

7. **Scope of v1.** Postcard mode first (hotels), vlog mode first (cabin), or both in parallel? Recommendation: Postcard first, since it exercises the whole pipeline with the shortest feedback loop.
8. **Repository.** This product should live in its own repo (e.g. `cinematic-editor`) rather than inside `trading-suite`; confirm and I'll scaffold it there.

---

8. Suggested repo layout (for Phase 0)

```
cinematic-editor/
  apps/
    web/            # Next.js PWA (desktop + phone)
    api/            # FastAPI: auth, catalog, looks, projects, EDL, jobs
    worker/         # ingest, label, transcribe, embed, beats, render
  packages/
    edl/            # EDL schema + validator (shared Python/TS types)
    looks/          # Look schema, LUT handling, camera profiles
    agent/          # edit-agent prompts, tools, style cards
  infra/
    docker-compose.yml   # postgres, redis, api, worker, web
  luts/             # technical + creative LUTs
  docs/
```

Part C — KICKOFF.md

Three pieces:

1. **CLAUDE.md** — paste into the root of the new repo so every session starts with the same context (section A).
2. **First prompt** — paste into a Claude Code Project thread to run Phase 0 and Phase 1 (section B).
3. **Follow-on prompts** — one per later phase (section C).

Attach `PLAN.md` from this folder to the Project (or commit it to the new repo under `docs/PLAN.md`) so Claude can read the full plan.

A. CLAUDE.md for the new repo

```
# Cinematic Editor

Prompt-driven editing system that turns a labeled library of real cinematic footage
into finished, color-graded videos. Full plan: docs/PLAN.md. Read it before any work.

## What we are building
- Postcard mode: 15–90 s music-driven, graded reels for hotels and travel.
- Vlog mode: 4–20 min transcript-driven narrative edits (cabin build, jewelry making).
- Desktop is the workstation. Phone (installable PWA) is for browsing, prompting,
  reviewing drafts, and approving renders.
- Real footage only. No AI-generated video in v1.

## Stack (do not change without asking)
- Backend: Python 3.12, FastAPI, SQLAlchemy, Alembic.
- Workers: Python, RQ on Redis. ffmpeg/ffprobe, PySceneDetect, faster-whisper,
  open_clip (SigLIP/CLIP), OpenCV, librosa, OpenTimelineIO.
- DB: Postgres 16 + pgvector.
- Frontend: Next.js (App Router), React, TypeScript, Tailwind, PWA, hls.js.
- AI: Anthropic SDK. Haiku-class model for bulk frame labeling, Opus/Fable-class for
  edit planning. Model IDs live in one config file, never hardcoded elsewhere.
- Infra: Docker Compose (postgres, redis, api, worker, web). Runs on the owner's desktop.
- Storage: local path or S3-compatible bucket behind one storage interface.

## Layout
apps/web, apps/api, apps/worker, packages/edl, packages/looks, packages/agent,
infra/, luts/, docs/

## Rules
- The EDL (packages/edl) is the contract between the edit agent and the renderer.
  Every change to it needs a schema bump, validator update, and a test.
- The edit agent may only reference shot IDs returned by its tools. The validator
  rejects anything else.
- Never run heavy jobs in the API process. Everything over ~1 s goes to a worker job
  with progress reporting.
- Proxies are 720p H.264 + HLS. Originals are never served to the phone.
- Every phase ends with a runnable demo and a short docs/phases/<n>.md note on what
  works, what does not, and how to run it.
- Tests: pytest for Python, vitest for web. Run them before every commit.
- Commit small, descriptive commits. Open a PR per phase.
- Ask before adding a paid service or a dependency over 50 MB.

## Owner's footage and preferences
```

```
(Fill in after Phase 0: cameras, log profiles, storage path, GPU, aspect ratios,
music library, finishing path, per-workspace style notes.)
```

B. First prompt (Phase 0 + Phase 1)

```
You are building the Cinematic Editor described in docs/PLAN.md. Read PLAN.md and
CLAUDE.md fully before doing anything.

My answers to the plan's open questions:
1. Cameras / log profiles: <e.g. Sony A7IV S-Log3, DJI Mini 4 Pro D-Log M, iPhone 15 Pro Apple Log>
2. Footage location and size: <e.g. /Volumes/Footage on the desktop, ~6 TB now, ~3 TB/year>
3. Desktop: <e.g. Mac Studio M2 Max 64 GB / Windows + RTX 4080>
4. Finishing: <ffmpeg output is fine for most; Resolve export for hero pieces>
5. Deliverables: <9:16 Reels first, 16:9 YouTube second; burned captions on vlogs>
6. Music: <Artlist subscription / none yet>
7. Start with: Postcard mode (hotels).
8. This is its own repo.

If any answer above is still a placeholder in angle brackets, use the plan's default
and note the assumption in docs/phases/0.md.

Do Phase 0 and Phase 1 from the plan:

Phase 0
- Scaffold the monorepo layout from PLAN.md section 8.
- Docker Compose with postgres (pgvector), redis, api, worker, web. `make up` brings
  everything up; `make test` runs all tests.
- Alembic migrations for the core tables in PLAN.md section 3.2.
- A storage interface with two backends: local filesystem and S3-compatible.
- Write docs/phases/0.md describing how to run it.

Phase 1 — ingest, catalog, search
- `cinematic ingest <path>` CLI and an API endpoint that enqueue ingest jobs.
- Per file: ffprobe metadata, PySceneDetect shots, 720p proxy + HLS, thumbnails and
  contact sheet per shot, faster-whisper transcript with word timestamps, quality
  scores (sharpness, clipping, stability), SigLIP embeddings per shot, near-duplicate
  grouping, vision labels via Claude using the controlled vocabulary in label_vocab.
- Camera profile detection from metadata and filename patterns for the cameras I
  listed; store the detected transfer function on the asset.
- Web app: library grid with filters (workspace, label, date, quality), text and
  semantic search, "find similar", favorites/rejects, batch tagging, shot detail with
  proxy playback. Must work at phone width.
- Tests for the EDL package skeleton, the ingest pipeline on a 30-second sample clip
  (generate a synthetic test clip with ffmpeg so tests need no real footage), and
  the search endpoints.
- docs/phases/1.md with a demo script.

Working style
- Work in small PRs, one per major piece (scaffold, migrations, ingest worker,
  labeling, web library). Tell me when each is ready to review.
- When you need my desktop (real footage, GPU timing), say so explicitly and give me
  the exact commands to run, or request a Remote Control thread.
- Stop and ask before adding any paid service beyond the Anthropic API.
- Phase 1 is done when I can type "misty lake drone shots, sharp, dawn" into the
  library and get the right shots from my footage.
```

C. Follow-on prompts

Phase 2 — Looks and grading

```
Read docs/PLAN.md sections 2.2 and 3.5 and docs/phases/1.md. Build Phase 2: the Look
model and editor, LUT upload (.cube), camera profiles with technical LUTs for my
cameras, reference-frame shot matching, ffmpeg grade pipeline on proxies and
originals, a compare-wipe preview, and batch re-grade as a worker job. Done when a
hotel Look applied across clips from all my cameras reads as one consistent piece.
Deliver docs/phases/2.md.
```

Phase 3 — Postcard mode

```
Read docs/PLAN.md sections 2.3, 3.3, 3.4 and docs/phases/2.md. Build Phase 3: music
library with librosa beat detection, the edit agent with catalog tools and the EDL
validator, proxy renderer and final renderer for 9:16, 16:9, 4:5 and 1:1 with
auto-reframe, title and end-card templates from the Look, the chat revision loop, and
edit version history. Done when one prompt produces a hotel reel I would post with at
most two revision rounds. Deliver docs/phases/3.md.
```

Phase 4 — Vlog mode

```
Read docs/PLAN.md section 2.4 and docs/phases/3.md. Build Phase 4: transcript-driven
select scoring, silence and filler removal, b-roll matching to speech, chapters,
burned and .srt captions, music ducking, intro/outro, and OTIO/FCPXML export to
DaVinci Resolve. Done when a week of cabin footage becomes a reviewable 10-minute cut
in under an hour of my time. Deliver docs/phases/4.md.
```

Phase 5 — Mobile review and remote access

```
Read docs/PLAN.md sections 2.5 and 3.6 and docs/phases/4.md. Build Phase 5: PWA
polish, HLS review player with timestamped comments that feed back into revision
prompts, approve/reject flow, web push on render completion, and a documented
Tailscale setup (plus optional R2 for proxies). Done when I can approve a render from
my phone away from home. Deliver docs/phases/5.md.
```